

Comprehensive Test Plan - Thermal Simulation Program

fs881 bta26

Legend for IDs

U = Unit test (single class or method)

I = Integration test (interfaces between classes)

S = System test (end-to-end behaviour)

E = Exceptional / invalid inputs (robustness)

Unit Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
U1	Create Component correctly	1) Create Component("MOSFET", 4.0, 100)	name="MOSFET", Rth=4.0, Tmax=100	Attributes stored exactly	Pass if attributes match inputs
U2	Component.is_safe returns SAFE	1) c=Component(..., Tmax=100) 2) call c.is_safe(89.9)	temperature=89.9, well below Tmax and not within 10C of limit	Returns "SAFE"	Pass if "SAFE"
U3	Component.is_safe returns WARNING when within 10C of limit	1) c=Component(..., Tmax=100) 2) call c.is_safe(95)	temperature=95, within 10C of Tmax	Returns "WARNING"	Pass if "WARNING"
U4	Component.is_safe boundary at exactly Tmax	1) c.is_safe(100)	temperature == Tmax	Returns "WARNING" (within 10C of limit)	Pass if "WARNING"
U5	Component.is_safe returns UNSAFE	1) c.is_safe(100.01)	temperature > Tmax	Returns "UNSAFE"	Pass if "UNSAFE"
U6	DutyCycle initialises cycle_time	1) d=DutyCycle(5,5,12)	on=5, off=5	cycle_time=10	Pass if cycle_time correct
U7	DutyCycle.power_at_time ON region start	1) d.power_at_time(0)	t=0, on=5/off=5	returns power_on	Pass if 12
U8	DutyCycle.power_at_time ON region inside	1) d.power_at_time(4.999)	t < on_time	returns power_on	Pass if 12
U9	DutyCycle.power_at_time boundary switch	1) d.power_at_time(5)	t == on_time	returns 0	Pass if 0

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
U10	DutyCycle.power_at_time OFF region	1) d.power_at_time(7)	$5 \leq t < 10$	returns 0	Pass if 0
U11	DutyCycle repeats correctly (mod behaviour)	1) d.power_at_time(12)	cycle_time=10, t%10=2	returns power_on	Pass if 12
U12	ThermalModel stores ambient+component	1) create Component 2) m=ThermalModel(25,c)	ambient=25	attributes stored	Pass if stored correctly
U13	ThermalModel.next_temperature steady-state target	1) target=T _a + P*R _{th}	T _a =25, P=10, R _{th} =4	target=65	Pass if computed target is 65
U14	ThermalModel.next_temperature increases when below target	1) current=25 2) call next_temperature(current, P>0, dt=1)	current < target	new_temp > current	Pass if temp rises
U15	ThermalModel.next_temperature decreases when above target	1) current=80 2) call with P=0, dt=1	target=ambient=25	new_temp < current	Pass if temp falls
U16	ThermalModel dt effect	1) compute next_temp with dt=0.5 vs dt=2.0	same current/P	dt=2 changes more than dt=0.5	Pass if larger dt causes larger change
U17	ResultsAnalyser max_temperatures basic	1) Provide small fake history	history=[(0,[25,25]),(1,[30,28])]	returns [30,28]	Pass if matches
U18	ResultsAnalyser max_temperatures with constant	1) history temps constant all 25	all 25	returns [25,...]	Pass if equals constants
U19	ResultsAnalyser max_temperatures empty history	1) history=[]	empty	Returns [0, 0, ...] with length equal to number of components	Pass if list of zeros returned and length matches number of components
U20	ResultsAnalyser thermal_margins nominal	1) components Tmax [100,90] 2) max_temps [30,50]	margin = Tmax - max	returns [70,40]	Pass if correct
U21	ResultsAnalyser thermal_margins negative margin	1) max_temp > Tmax	unsafe	margin negative	Pass if negative values appear correctly

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
U22	ScenarioManager.plot handles 1 scenario	1) add one scenario 2) plot	one dataset	plot appears with one line and label	Pass if plot renders without error
U23	ScenarioManager.plot handles multiple scenarios	1) add 2+ scenarios	multiple datasets	plot shows multiple labelled lines	Pass if plot renders and legend contains both names
U24	ScenarioManager.show_results_figure SAFE row	1) run low-power scenario 2) call show_results_figure()	peak temp well below limit	row displays SAFE status with green colour	Pass if status column shows SAFE
U25	ScenarioManager.show_results_figure WARNING row	1) run scenario where peak is within 10C of limit 2) call show_results_figure()	peak temp within 10C of max_temperature	row displays WARNING status and warning message shown below table	Pass if WARNING appears and warning message is displayed
U26	ScenarioManager.show_results_figure UNSAFE row	1) run high-power scenario causing overtemperature 2) call show_results_figure()	peak temp exceeds max_temperature	row displays UNSAFE status with red colour	Pass if UNSAFE appears in status column
U27	ScenarioManager.show_results_figure result row Completed	1) run safe scenario 2) call show_results_figure()	stop_reason = Completed	result row shows Completed in green	Pass if Completed shown
U28	ScenarioManager.show_results_figure result row Overtemperature	1) run unsafe scenario 2) call show_results_figure()	stop_reason = Overtemperature	result row shows overtemperature component name and overtemp_time	Pass if overtemp details shown correctly

Integration Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
I1	Simulation initialises temperature list correctly	1) Create 2 components + 2 models 2) Run sim for 0 seconds	duration=0	history has at least one entry at t=0, temps list length=2	Pass if history contains (0, [T1,T2]) and len temps matches
I2	Simulation updates all components each step	1) Create 2 models with different Rth 2) Run multiple steps	same duty cycle P	component temps diverge due to different Rth	Pass if temps differ after some steps
I3	Simulation stores history as tuples	1) Run sim 2) check history format	history entries	each entry is (t, [temps])	Pass if structure matches
I4	Simulation uses copy of temperatures	1) Run sim 2) check earlier history temps do not change later	copy behaviour	earlier entries remain unchanged	Pass if earlier temps stable (not overwritten)
I5	Simulation stop_reason Completed	1) Choose low power and high Tmax 2) run full duration	safe all run	stop_reason="Completed", overtemp_time=None	Pass if Completed and full duration reached
I6	Simulation records overtemperature but does NOT stop early	1) Choose high power, low Tmax 2) run full duration	unsafe occurs mid-simulation	stop_reason starts with "Overtemperature:", overtemp_time recorded, history runs to full duration	Pass if full history length matches duration/timestep and overtemp_time is not None
I7	Simulation overtemp_time recorded correctly	1) Lower Tmax so overtemp occurs at known time 2) run	known overtemp time	overtemp_time matches expected time step	Pass if overtemp_time value is correct
I8	Safety check identifies which component failed first	1) Make component A low Tmax 2) component B high Tmax	A should fail first	stop_reason contains A name	Pass if correct component name appears
I9	is_safe used consistently in Simulation and show_results_figure	1) Run scenario 2) check stop_reason 3) check status in results figure	same component, same peak temp	Both use is_safe() and agree on UNSAFE status	Pass if both flag same component as UNSAFE
I10	ScenarioManager stores results structure	1) run_scenario("A", sim, comps)	normal run	results_db["A"] has keys: history, max_temperatures, thermal_margins, stop_reason, overtemp_time	Pass if all keys exist and types correct

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
I11	ScenarioManager overwriting scenario name	1) run_scenario("A", sim1) then run_scenario("A", sim2)	same key	latest results overwrite	Pass if results_db["A"] equals second run
I12	ScenarioManager multiple scenarios	1) run "A" and "B"	2 keys	results_db has both	Pass if both present and distinct
I13	show_results_figure does not call plt.show()	1) call show_results_figure() alone without plt.show()	no plt.show in method	No window opens until plt.show() is called separately at script end	Pass if window only opens after external plt.show()
I14	plot() does not call plt.show()	1) call plot() alone without plt.show()	no plt.show in method	No window opens until plt.show() is called separately at script end	Pass if window only opens after external plt.show()
I15	Both figures open simultaneously	1) call show_results_figure() 2) call plot() 3) call plt.show()	single plt.show at end	Both figure windows open at the same time	Pass if both windows appear together without blocking

System Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
S1	End-to-end: single component, 50% duty	1) Create 1 component 2) Create model list with 1 model 3) duty 5/5, P=12 4) run scenario manager	nominal	results_db filled, stop_reason Completed, overtemp_time=None	Pass if no crash and outputs plausible
S2	End-to-end: compare 50% vs 80% duty	1) scenario A 5/5 2) scenario B 8/2 3) plot	two scenarios	higher duty shows higher temps	Pass if B curve above A most of time
S3	End-to-end: overtemperature scenario runs to completion	1) lower Tmax to 40 2) run full duration	unsafe	stop_reason indicates overtemp, overtemp_time recorded, history runs to full duration	Pass if full history and overtemp details correct
S4	End-to-end: multi-component list behaviour	1) 3 components 3 models list 2) run	list handling	history stores temps list length=3; max_temps length=3; margins length=3	Pass if all lengths match number of components

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
S5	End-to-end: results consistency	1) run scenario 2) compute max_temps 3) verify each max temp equals max over history temps[i]	consistency	values align	Pass if computed max equals direct max from history
S6	End-to-end: show_results_figure and plot open together	1) run both scenarios 2) call show_results_figure() 3) call plot() 4) call plt.show()	two figure windows	Both windows appear simultaneously, neither blocks the other	Pass if both windows open at same time and close independently
S7	End-to-end: WARNING appears in results figure for near-limit component	1) tune power so peak is within 10C of limit 2) call show_results_figure()	near-limit scenario	WARNING status shown and warning message displayed below table	Pass if WARNING and message both present
S8	End-to-end: separate component lists per scenario	1) run two scenarios 2) check temperatures at t=0 for both	independent state	Both scenarios start from ambient temperature independently	Pass if no state shared between scenarios

Exceptional / Invalid Input Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
E1	DutyCycle with on_time=0	1) DutyCycle(0,5,12) 2) power_at_time(0..10)	on=0	always OFF	Pass if always returns 0
E2	DutyCycle with off_time=0	1) DutyCycle(5,0,12) 2) power_at_time(0..10)	off=0	always ON	Pass if always returns 12
E3	DutyCycle invalid negative times	1) DutyCycle(-1,5,12)	on_time negative	No exception raised. Power switching executes but behaviour is non-physical	Pass if no crash and behaviour documented as invalid parameter case

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
E4	DutyCycle cycle_time=0 case	1) DutyCycle(0,0,12) 2) call power_at_time(1)	cycle_time=0	Raises ZeroDivisionError when calling power_at_time(t) due to modulo by zero	Pass if ZeroDivisionError occurs
E5	Simulation with time_step=0	1) run Simulation(..., time_step=0)	dt=0	Program raises ValueError or enters infinite loop. Should raise ValueError indicating invalid time step.	Pass if error raised and no infinite loop
E6	Simulation with negative duration	1) duration=-1	invalid	Returns empty history [] and stop_reason "Completed"	Pass if history empty and reason is Completed
E7	ThermalModel with negative thermal_resistance	1) Rth=-2	physically invalid	No exception raised. Temperature trend may be non-physical.	Pass if code runs without crashing and behaviour documented as invalid parameter case
E8	ScenarioManager.plot with empty results_db	1) create manager 2) call plot()	no scenarios	Displays an empty plot; may show a Matplotlib legend warning but does not crash.	Pass if no exception occurs and plot window appears
E9	ScenarioManager.show_results_figure with empty results_db	1) create manager 2) call show_results_figure()	no scenarios	Displays an empty table figure without crashing	Pass if no exception occurs and figure window appears
E10	Component.is_safe at exactly 10C below limit	1) c=Component(..., Tmax=100) 2) c.is_safe(90)	temperature = Tmax - 10	Returns "WARNING" (boundary of warning zone)	Pass if "WARNING"
E11	Component.is_safe at exactly 10C + 0.01 below limit	1) c.is_safe(89.99)	temperature just outside warning zone	Returns "SAFE"	Pass if "SAFE"